In [1]:
```
# Tuples : ordered, immutable, allows duplicate values

# Tuples may be constructed in a number of ways:

# Using a pair of parentheses to denote the empty tuple: ()

# Using a trailing comma for a singleton tuple: a, or (a,)

# Separating items with commas: a, b, c or (a, b, c)

# Using the tuple() built-in: tuple() or tuple(iterable)


myTuple0 = ()
print(f"Type: {type(myTuple0)}, myTuple0: {myTuple0}")

# to create a tuple with only one element, add comma after the element
myTuple1 = ('a',)
print(f"Type: {type(myTuple1)}, myTuple1: {myTuple1}")

myTuple2 = 'a', 'b', 'c'
print(f"Type: {type(myTuple2)}, myTuple2: {myTuple2}")

myTuple3 = ('a', 'b', 'c')
print(f"Type: {type(myTuple3)}, myTuple3: {myTuple3}")

# allows duplicate values
myTuple4 = ('a', 'b', 'b', 'c')
print(f"Type: {type(myTuple4)}, myTuple4: {myTuple4}")
```

```
Type: <class 'tuple'>, myTuple0: ()
Type: <class 'tuple'>, myTuple1: ('a',)
Type: <class 'tuple'>, myTuple2: ('a', 'b', 'c')
Type: <class 'tuple'>, myTuple3: ('a', 'b', 'c')
Type: <class 'tuple'>, myTuple4: ('a', 'b', 'b', 'c')
```

In [3]:
```
# tuple() - to create a new tupple
# with no argument, tuple() creates an empty tuple

myTuple = tuple()
print(f"Type: {type(myTuple)}, myTuple: {myTuple}")
```

```
Type: <class 'tuple'>, myTuple: ()
```

In [4]:
```
"""
if the argument is a sequence,
the result of the call to tuple is a tuple
with the elements of the sequence
"""
myTuple = tuple("mca")
print(f"Type: {type(myTuple)}, myTuple: {myTuple}")
```

```
Type: <class 'tuple'>, myTuple: ('m', 'c', 'a')
```

In [5]:
```
myTuple = tuple(["bca", "mca"])
print(f"Type: {type(myTuple)} myTuple: {myTuple}")
```

```
Type: <class 'tuple'> myTuple: ('bca', 'mca')
```

In [6]:
```
myTuple = tuple(("bca", "mca"))
print(f"Type: {type(myTuple)}, myTuple: {myTuple}")
```

```
Type: <class 'tuple'>, myTuple: ('bca', 'mca')
```

In [7]:
```
# Tuples may be nested

student = 5, 'Animesh', 'mca'
studentMarks = student, (97, 90, 73, 85)
print(studentMarks)
```

```
((5, 'Animesh', 'mca'), (97, 90, 73, 85))
```

In [8]:
```
# len(tuple) - number of itemts in the tuple

courses = ("bca", "mca", "btech", "bsc")
print(len(courses))
t = tuple()
print(len(t))
```

```
4
0
```

In [9]:
```
# like list, tuple items can be accessed by referring to the index number within square brackets
# like list, slicing selects a range of items of the tuple
# like list, membership of an element can be checked with 'in' or 'not in'
# like list, loops like for and while work on a tuple
# like list concatenation with + and duplication with * works on tuple
```

In [10]:
```
# tuples are immutable; Once a tuple is created,
# its individual values cannot be changed

myTuple = 'a', 'b', 'c', 'd'
myTuple[0] = 'A'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[10], line 5
      1 # tuples are immutable; Once a tuple is created,
      2 # its individual values cannot be changed
      4 myTuple = 'a', 'b', 'c', 'd'
----> 5 myTuple[0] = 'A'

TypeError: 'tuple' object does not support item assignment
```

In [11]:
```
# tuples are immutable; Once a tuple is created,
# its individual values cannot be changed
# Workaround 1: replace one tuple with another

myTuple = 'a', 'b', 'c', 'd'

myTuple = ('A',) + myTuple[1:]

print(f"Type: {type(myTuple)}, myTuple: {myTuple}")
```

```
Type: <class 'tuple'>, myTuple: ('A', 'b', 'c', 'd')
```

In [12]:
```
# tuples are immutable; Once a tuple is created,
# its individual values cannot be changed
# Workaround 2: convert the tuple into a list,
# change the list, and convert the list back into a tuple

myTuple = 'a', 'b', 'c', 'd'

myList = list(myTuple)
myList[0] = 'A'
myTuple = tuple(myList)

print(f"Type: {type(myTuple)}, myTuple: {myTuple}")
```

```
Type: <class 'tuple'>, myTuple: ('A', 'b', 'c', 'd')
```

In [13]:
```
# It is not possible to assign to the individual items of a tuple,
# however it is possible to create tuples which contain
# mutable objects, such as lists.

t = 1, 2, [3, 4, 5], 6
t
```

Out[13]:
```
(1, 2, [3, 4, 5], 6)
```

In [14]:
```
# It is not possible to assign to the individual items of a tuple
t = 1, 2, [3, 4, 5], 6
t[2] = [9, 10]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 3
      1 # It is not possible to assign to the individual items of a tuple
      2 t = 1, 2, [3, 4, 5], 6
----> 3 t[2] = [9, 10]

TypeError: 'tuple' object does not support item assignment
```

In [15]:
```
# But the elements of the mutable list inside the tuple can be changed
t = 1, 2, [3, 4, 5], 6
t[2][1] = 44
t
```

Out[15]:
```
(1, 2, [3, 44, 5], 6)
```

In [16]:
```
# delete a tuple

myTuple = 'a', 'b', 'c', 'd'
del myTuple
print(myTuple)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[16], line 5
      3 myTuple = 'a', 'b', 'c', 'd'
      4 del myTuple
----> 5 print(myTuple)

NameError: name 'myTuple' is not defined
```

In [17]:
```
# comparing tuples
# starts by comparing the first element from each tuple.
# if they are equal, goes on to check the next element,
# and so on, until elements are found which differ.

(1, 2, 3) < (1, 3, 1)
```

Out[17]:
```
True
```

In [18]:
```
(0, 100, 1000) < (1, 0, 0)
```

Out[18]:
```
True
```

In [19]:
```
# unpack tuples - extract values to variables
# unpacking requires that there are as many variables
# on the left side of the equal sign as there are elements in the tuple.

student = (5, 'Animesh', 'mca') # packing

(roll, name, degree) = student  # unpacking

print(f"roll: {roll}, name: {name}, degree: {degree}")
```

```
roll: 5, name: Animesh, degree: mca
```

In [20]:
```
courses = ('bca','mca', 'msc', 'mcom','diploma')

(ug,*pg,other) = courses

print(f"ug: {ug}, pg: {pg}, other: {other}")
```

```
ug: bca, pg: ['mca', 'msc', 'mcom'], other: diploma
```

In [21]:
```
# the right side can be any kind of sequence

email_id = 'mca.students@heritageit.edu'
uname, domain = email_id.split('@')
a,b,c =[1,2,3]
print(uname)
print(domain)
print(a,b,c)
```

```
mca.students
heritageit.edu
1 2 3
```

In [22]:
```
print(dir(tuple))
```

```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge
__', '__getattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '__
le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__rmul__', '__setattr_
_', '__sizeof__', '__str__', '__subclasshook__', 'count', 'index']
```

In [23]:
```
help(tuple.count)
```

```
Help on method_descriptor:

count(self, value, /)
    Return number of occurrences of value.
```

In [24]:
```
# count() method returns the number of times a specified value appears in the tuple.

t = (5, 6, 7, 4, 7, 4, 4, 5, 8, 5)
n = t.count(5)
print(n)
```

```
3
```

In [25]:
```
help(tuple.index)
```

```
Help on method_descriptor:

index(self, value, start=0, stop=9223372036854775807, /)
    Return first index of value.

    Raises ValueError if the value is not present.
```

In [26]:
```
# index() method finds the first occurrence of the specified value
# raises an exception if the value is not found

t = (5, 6, 7, 4, 7, 4, 4, 5, 8, 5)
n = t.index(7)
print(n)
```

```
2
```

In [27]:
```
# Given a list of tuples, each containing name, age, and profession,
# generate strings of English sentences as
# "Name is X years old and works as Y." for each one.
# Sample Input: [('Adnan', 25, "Programmer"), ("Swati", 27, 'DBA'), ('Anand', 26, "Tester")]
# Output: Adnan is 25 years old and works as Programmer.
# Swati is 27 years old and works as DBA.
# Anand is 26 years old and works as Tester.
# Roll:14
employees = [('Adnan', 25, "Programmer"), ("Swati", 27, 'DBA'), ('Anand', 26, "Tester")]
for employee in employees:
    name,years,designation = employee
    print(f"{name} is {years} old and works as {designation}.")
```

```
Adnan is 25 old and works as Programmer.
Swati is 27 old and works as DBA.
Anand is 26 old and works as Tester.
```

In [28]:

```python
# Create a list of tuples from given list of numbers,
# with number and its square in each tuple.
# Sample Input: [2, 5, 1, 4]
# Output: [(2, 4), (5, 25), (1, 1), (4, 16)]
# Roll: 02
x = [2, 5, 1, 4]
squared = []
for i in x:
    squared.append((i,i**2))
print(squared)
```

```
[(2, 4), (5, 25), (1, 1), (4, 16)]
```